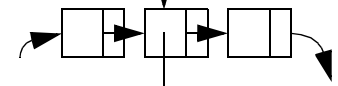
Education Data Structure

EDUCATION

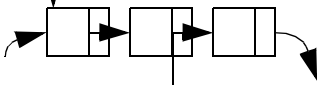
```
/* Usage */
EDUCATION *education_fetch(
        season_name,
        year,
        spreadsheet_filename,
        date_label );

/* Input */
SEMESTER *semester =
        semester_fetch(
            season_name,
            year,
            fetch_offering_list,
            fetch_registration_list );

PAYPAL *paypal =
        paypal_fetch(
            spreadsheet_filename,
            date_label );

LIST *education_program_list() =
        program_list(
            1 /* fetch_alias_list */ );

/* Process */
double education_net_revenue(
        transaction_amount,
        merchant_transaction_fee );

LIST *education_paypal_list(
        spreadsheet_filename,
        paypal->spreadsheet,
        paypal->paypal_dataset,
        semester_offering_list(),
        program_list(),
        product_list(),
        event_list() );

/* Returns paypal_dataset */
PAYPAL_DATASET
*education_paypal_dataset(
        input_string,
        paypal->
            spreadsheet->
            spreadsheet_column_list(),
        paypaypal>paypal_dataset );

PAYPAL_DEPOSIT *
education_paypal_deposit(
        season_name,
        year,
        semester_offering_list(),
        education_program_list(),
        education_product_list(),
        paypal_dataset,
        row_number );

/* Output */

void education_paypal_insert(
        education_paypal_list() );

void education_payment_insert(
        education_paypal_list() );

void education_enrollment_insert(
        education_paypal_list() );
```

SEMESTER

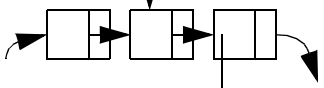
```
/* Usage */
SEMESTER *semester_new(
        char *season_name,
        int year );

/* Process */
LIST *semester_offering_list(
        season_name,
        year );

LIST *semester_event_list(
        season_name,
        year );
```



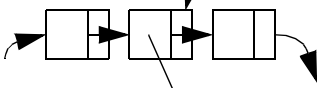
OFFERING



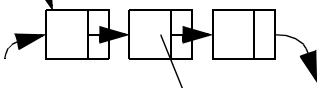
EVENT



PAYPAL_DEPOSIT



PRODUCT



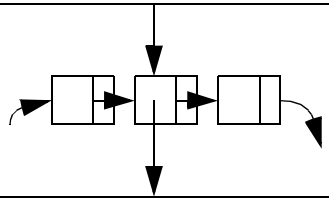
PROGRAM

PAYPAL_DEPOSIT

PAYPAL_DATASET

PAYPAL_DEPOSIT (1)

```
/* Usage */
PAYPAL_DEPOSIT *paypal_deposit_fetch(
        payor_full_name,
        payor_street_address,
        season_name,
        year,
        deposit_date_time,
        fetch_tuition_payment_list,
        fetch_program_payment_list,
        fetch_tuition_refund_list,
        fetch_product_refund_list,
        fetch_sweep );

/* Input */
ENTITY *payor_entity =
        entity_new(
            payor_full_name,
            payor_street_address );

SEMESTER *semester =
        semester_new(
            season_name,
            year );

char *paypal_date_time;
double paypal_amount;
double transaction_fee;
double account_balance;
char *transaction_ID;
char *invoice_number;
char *from_email_address;

/* Process */
int row_number;

LIST *paypal_item_list =
paypal_item_list(
        paypal_item_entity_delimit(
            education_paypal_dataset()->
                item_title_P ),
        education_paypal_dataset()->
            transaction_type_E,
        education_paypal_allowed_list(
            offering_name_list()
            program_name_list(),
            program_alias_name_list(),
            product_name_list(),
            event_list() ) );

void deposit_set_paypal_item_expected_revenue(
        paypal_item_list(),
        semester_offering_list() );
```

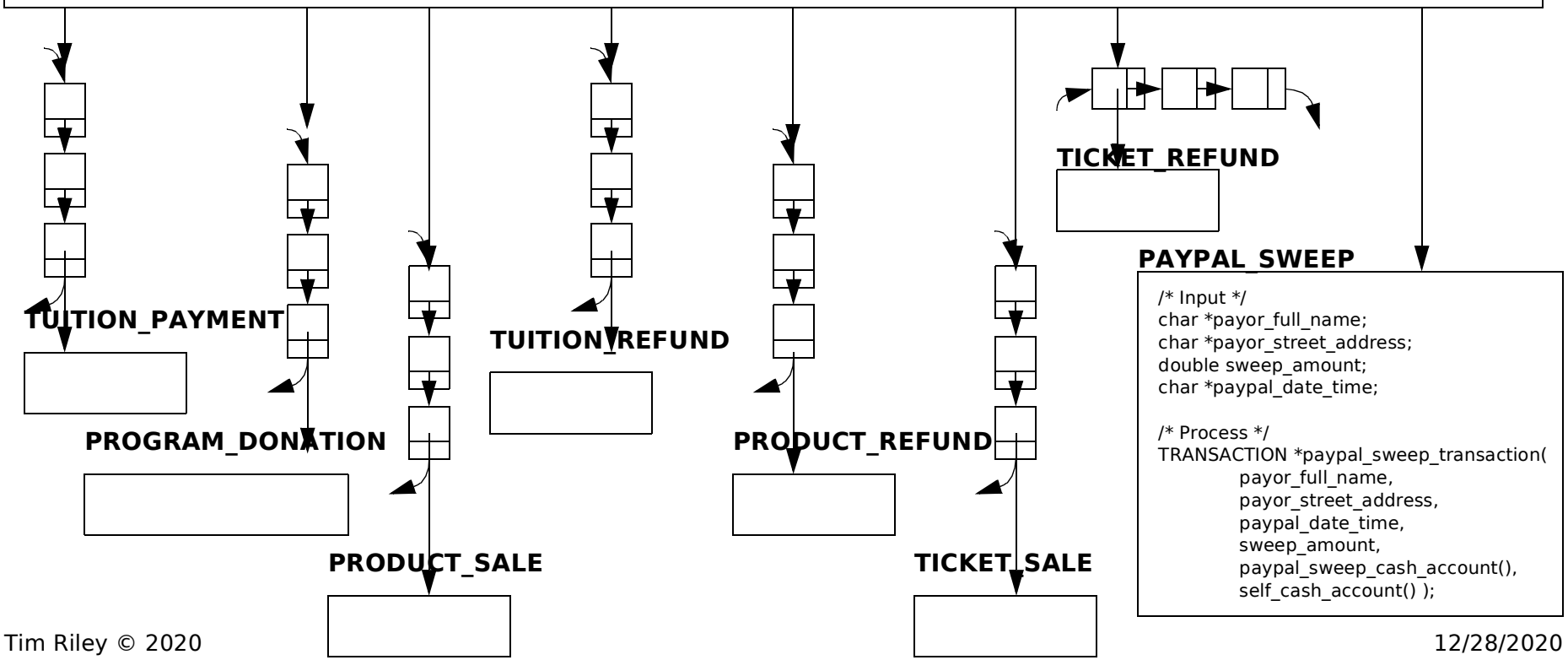


PAYPAL_ITEM

```
/* Usage */
LIST *paypal_item_list(
        not_exists_revenue_item_list,
        entity_delimited_item_title_P,
        transaction_type_E,
        allowed_item_list );

/* Input */
double expected_revenue;

/* Process */
char *item_data;
ENTITY *benefit_entity;

double paypal_item_value(
        deposit_amount,
        this->expected_revenue,
        paypal_item_expected_revenue_total(),
        paypal_item_nonexpected_revenue_length() );

double paypal_item_fee(
        deposit_amount,
        transaction_fee,
        paypal_item_value() );

double paypal_item_gain(
        deposit_amount,
        paypal_item_expected_revenue_total(),
        paypal_item_nonexpected_revenue_length(),
        paypal_item_expected_revenue_length() );

LIST *paypal_item_steady_state_list(
        paypal_item_list,
        deposit_amount,
        transaction_fee,
        paypal_item_expected_revenue_total(),
        paypal_item_nonexpected_revenue_length(),
        paypal_item_expected_revenue_length() );
```

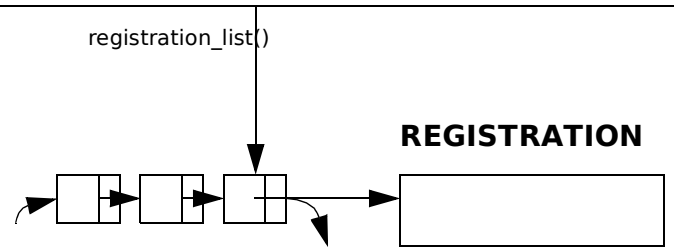
PAYPAL_DEPOSIT (2)

```
/* Process */

LIST *tuition_payment_list(
        season_name,
        year,
        paypal_item_steady_state_list(),
        semester_offering_list(),
        paypal_deposit );

LIST *program_donation_list(
        paypal_item_steady_state_list(),
        program_name_list(),
        program_alias_name_list(),
        paypal_deposit );

LIST *product_sale_list(
        paypal_item_steady_state_list(),
        product_name_list(),
        paypal_deposit );

LIST *tuition_refund_list(
        season_name,
        year,
        paypal_item_steady_state_list(),
        offering_name_list(),
        paypal_deposit );

LIST *product_refund_list(
        paypal_item_steady_state_list(),
        product_name_list(),
        paypal_deposit );

LIST *ticket_sale_list(
        paypal_item_steady_state_list(),
        event_list(),
        paypal_deposit );

LIST *ticket_refund_list(
        paypal_item_steady_state_list(),
        event_list(),
        paypal_deposit );

PAYPAL_SWEEP *paypal_sweep(
        paypal_deposit );
```



PAYPAL_DEPOSIT (3)

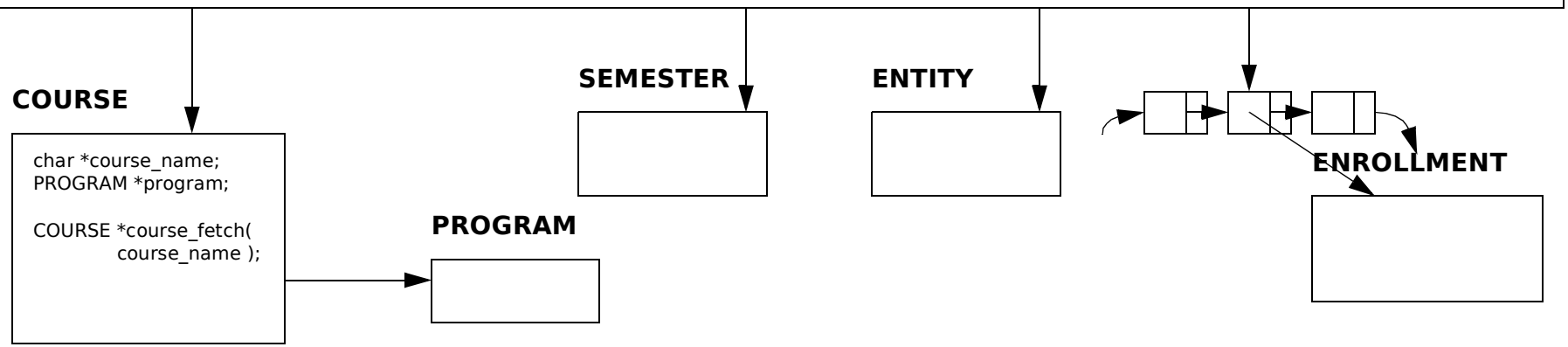
```
LIST *paypal_deposit_registration_list(
        deposit_tuition_payment_list(),
        deposit_tuition_refund_list() );

double tuition_payment_total(
        tuition_payment_list() );

double program_payment_total(
        program_payment_list() );

double product_payment_total(
        product_payment_list() );

double tuition_refund_total(
        tuition_refund_list() );

double product_refund_total(
        product_refund_list() );

double ticket_sale_total(
        ticket_sale_list() );

double ticket_refund_total(
        ticket_refund_list() );

double deposit_registration_tuition(
        deposit_registration_list() )
{
        return
            sum(
            A->
            registration->
            registration_tuition() );
}

double deposit_gain_donation(
        deposit_amount,
        deposit_registration_tuition() );

double deposit_net_revenue(
        deposit_amount,
        transaction_fee )
{
        return A - B;
}

double deposit_overpayment_loss(
        deposit_amount,
        deposit_registration_tuition() );

DEPOSIT *deposit_steady_state(
        this,
        semester_offering_list );

void deposit_insert(
        payor_full_name,
        payor_street_address,
        season_name,
        year,
        deposit_date_time,
        tuition_payment_total(),
        program_payment_total(),
        product_payment_total(),
        tuition_refund_total(),
        product_refund_total(),
        ticket_sale_total(),
        ticket_refund_total(),
        sweep_total(),
        transaction_fee );
```



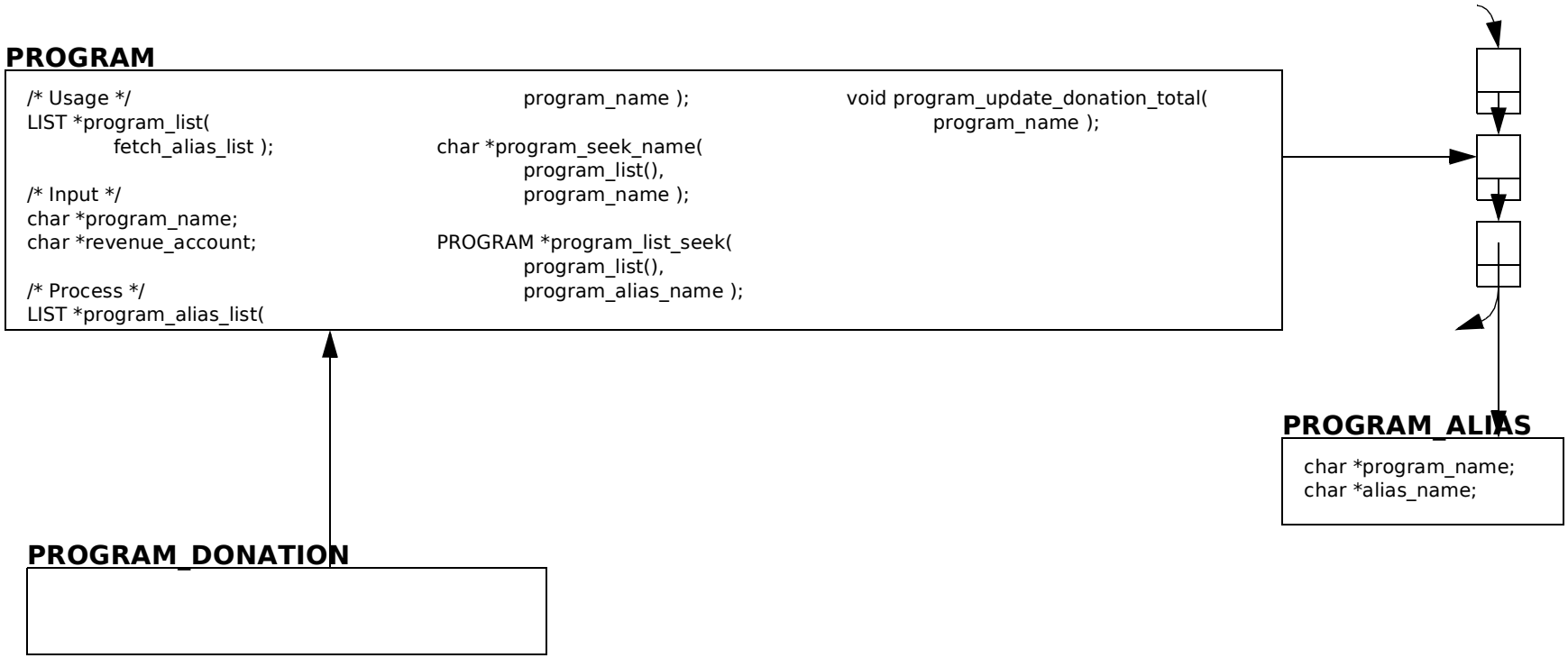
Education Data Structure

OFFERING

```
/* Usage */
OFFERING *offering_fetch(
        course_name,
        season_name,
        year,
        fetch_course,
        fetch_enrollment_list );

/* Input */
COURSE *course =
        course_fetch(
            course_name );

SEMESTER *semester =
        semester_new(
            season_name,
            year );

double course_price;
int class_capacity;
char *revenue_account;

ENTITY *instructor_entity =
        entity_new(
            instructor_full_name;
            instructor_street_address );

LIST *offering_enrollment_list(
        course_name,
        season_name,
        year );

/* Process */
OFFERING *offering_parse(
        input,
        fetch_course,
        fetch_enrollment_list )

int offering_enrollment_count(
        offering_enrollment_list() )
{
        return length( A );
}

int offering_capacity_available(
        class_capacity,
        offering_enrollment_count() )
{
        return A - B;
}

OFFERING *offering_steady_state(
        this,
        offering_enrollment_list() );

/* Output */
void offering_update(
        offering_enrollment_count(),
        offering_capacity_available(),
        course_name,
        season_name,
        year );
```



Education Data Structure



PROGRAM

PROGRAM_DONATION

```
/* Usage */
PROGRAM_DONATION *program_donation_fetch(
        program_name,
        payor_full_name,
        payor_street_address,
        season_name,
        year,
        payment_date_time,
        fetch_program,
        fetch_paypal );

/* Input */
PROGRAM *program;
double donation_amount;
double merchant_fees_expense;
char *paypal_date_time;

/* Process */
double net_payment_amount =
        education_net_payment_amount(
            donation_amount,
            merchant_fees_expense );

PROGRAM_DONATION *program_donation_steady_state(
        this,
        donation_amount,
        merchant_fees_expense );

PROGRAM_DONATION *program_donation_parse(
        input,
        fetch_program,
        fetch_paypal );

TRANSACTION *program_donation_transaction(
        payor_full_name,
        payor_street_address,
        payment_date_time,
        program_name,
        donation_amount,
        merchant_fees_expense,
        education_net_payment_amount(),
        entity_self_paypal_cash_account_name(),
        account_fees_expense(),
        program->revenue_account );

/* Output */
void program_payment_update(
        education_net_payment_amount(),
        program_donation_transaction()->
            transaction_date_time,
        program_name,
        payor_full_name,
        payor_street_address,
        season_name,
        year,
        payment_date_time );
```

TRANSACTION

Education Data Structure

PRODUCT

```
/* Usage */
PRODUCT *product_fetch(
        char *product_name,
        boolean fetch_program,
        boolean fetch_sale_list,
        boolean fetch_refund_list );

/* Input */
PROGRAM *program;

LIST *product_sale_list();
LIST *product_refund_list();

/* Process */
double product_sale_total(
        product_sale_list() );

double product_refund_total(
        product_refund_list() );

void product_update(
        product_sale_total(),
        product_refund_total(),
        product_name );
```

PRODUCT_SALE

```
/* Usage */
PRODUCT_SALE *product_sale_fetch(
        product_name,
        sale_date_time,
        payor_full_name,
        payor_street_address,
        fetch_product,
        fetch_paypal );

/* Input */
PRODUCT *product;
int quantity;
double retail_price;
double merchant_fees_expense;
char *paypal_date_time;

/* Process */
double product_sale_extended_price(
        quantity,
        retail_price );

double net_payment_amount =
        education_net_payment_amount(
        product_sale_extended_price(),
        merchant_fees_expense );

PRODUCT_SALE *product_sale_parse(
        input,
        fetch_product,
        fetch_paypal );

PRODUCT_SALE *product_sale_steady_state(
        this,
        quantity,
        retail_price,
        merchant_fees_expense );

TRANSACTION *product_sale_transaction(
        payor_full_name,
        payor_street_address,
        sale_date_time,
        product_name,
        program_name,
        product_sale_extended_price(),
        merchant_fees_expense,
        education_net_payment_amount(),
        entity_self_paypal_cash_account_name(),
        account_fees_expense(),
        product->revenue_account;

/* Output */
void product_sale_update(
        product_sale_extended_price(),
        education_net_payment_amount(),
        product_sale_transaction()->
            transaction_date_time,
        product_name,
        sale_date_time,
        payor_full_name,
        payor_street_address,
        season_name,
        year );
```

TRANSACTION

PRODUCT_SALE

PRODUCT_REFUND

```
/* Usage */
PRODUCT_REFUND *product_refund_fetch(
        product_name,
        sale_date_time,
        payor_full_name,
        payor_street_address,
        fetch_sale,
        fetch_paypal );

/* Input */
PRODUCT_SALE *product_sale;
char *refund_date_time;
double refund_amount;
double merchant_fees_expense;
char *paypal_date_time;

/* Process */
double net_refund_amount =
        education_net_refund_amount(
            refund_amount,
            merchant_fees_expense );

PRODUCT_REFUND *product_refund_parse(
        input,
        fetch_sale,
        fetch_paypal );

PRODUCT_REFUND *product_refund_steady_state(
        this,
        refund_amount,
        merchant_fees_expense );

TRANSACTION *product_refund_transaction(
        payor_full_name,
        payor_street_address,
        this->product_sale->
            program_name,
        refund_amount,
        merchant_fees_expense,
        education_net_payment_amount(),
        entity_self_paypal_cash_account_name(),
        account_fees_expense(),
        product->revenue_account );

/* Output */
void product_refund_update(
        education_net_refund_amount(),
        product_refund_transaction()->
            transaction_date_time,
        product_name,
        sale_date_time,
        payor_full_name,
        payor_street_address,
        refund_date_time );
```

TRANSACTION

Education Data Structure

EVENT

```
/* Usage */
EVENT *event_fetch(
        char *event_name,
        char *event_date,
        char *event_time,
        boolean fetch_program,
        boolean fetch_venue,
        boolean fetch_sale_list,
        boolean fetch_refund_list );

/* Input */
PROGRAM *program;
VENUE *venue;
LIST *ticket_sale_list();
LIST *ticket_refund_list();

/* Process */
int ticket_sale_count =
        ticket_sale_count(
            ticket_sale_list );

double ticket_sale_total =
        ticket_sale_total(
            ticket_sale_list );

int ticket_refund_count =
        ticket_refund_count(
            ticket_refund_list );

double ticket_refund_total =
        ticket_refund_total(
            ticket_refund_list );

int event_capacity_available(
        this->venue->capacity,
        ticket_sale_count(),
        ticket_refund_count() );

void event_update(
        ticket_sale_count(),
        ticket_sale_total(),
        ticket_refund_total(),
        event_capacity_available(),
        event_name,
        event_date,
        event_time );
```

TICKET_SALE

```
/* Usage */
TICKET_SALE *ticket_sale_fetch(
        event_name,
        event_date,
        event_time,
        sale_date_time,
        payor_full_name,
        payor_street_address,
        fetch_event,
        fetch_paypal );

/* Input */
EVENT *event;
int quantity;
double ticket_price;
double merchant_fees_expense;
char *paypal_date_time;

/* Process */
double ticket_sale_extended_price(
        quantity,
        ticket_price );

double net_payment_amount =
        education_net_payment_amount(
            ticket_sale_extended_price(),
            merchant_fees_expense );

TICKET_SALE *ticket_sale_parse(
        input,
        fetch_event,
        fetch_paypal );

TICKET_SALE *ticket_sale_steady_state(
        this,
        quantity,
        ticket_price,
        merchant_fees_expense );

TRANSACTION *ticket_sale_transaction(
        payor_full_name,
        payor_street_address,
        sale_date_time,
        event_name,
        event->program->program_name,
        ticket_sale_extended_price(),
        merchant_fees_expense,
        education_net_payment_amount(),
        entity_self_paypal_cash_account_name(),
        account_fees_expense(),
        event->revenue_account );

/* Output */
void ticket_sale_update(
        ticket_sale_extended_price(),
        education_net_payment_amount(),
        ticket_sale_transaction()->
            transaction_date_time,
        event_name,
        event_date,
        event_time,
        sale_date_time,
        payor_full_name,
        payor_street_address );
```

TRANSACTION

TICKET_SALE

TICKET_REFUND

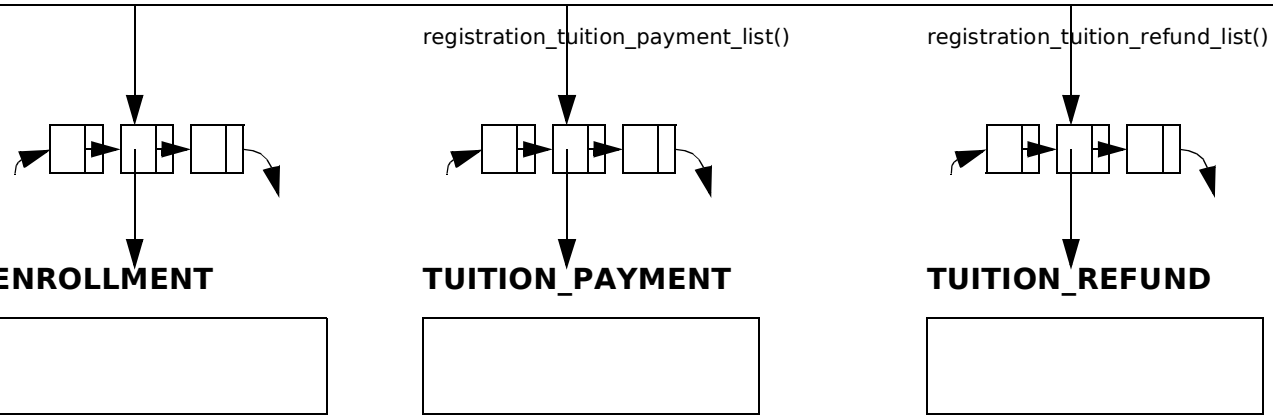
```
/* Usage */
TICKET_REFUND *ticket_refund_fetch(
        event_name,
        event_date,
        event_time,
        sale_date_time,
        payor_full_name,
        payor_street_address,
        refund_date_time,
        fetch_sale,
        fetch_paypal );

/* Input */
TICKET_SALE *ticket_sale;
char *refund_date_time;
double refund_amount;
double merchant_fees_expense;
char *paypal_date_time;

/* Process */
double net_refund_amount =
        education_net_refund_amount(
            refund_amount,
            merchant_fees_expense );

TICKET_REFUND *ticket_refund_parse(
        input,
        fetch_sale,
        fetch_paypal );

TICKET_REFUND *ticket_refund_steady_state(
        this,
        refund_amount,
        merchant_fees_expense );

TRANSACTION *ticket_refund_transaction(
        payor_full_name,
        payor_street_address,
        refund_date_time,
        this->
            ticket_sale->
            event->
            event_name,
        this->
            ticket_sale->
            event->
            program->
            program_name,
        refund_amount,
        merchant_fees_expense,
        education_net_refund_amount(),
        entity_self_paypal_cash_account_name(),
        account_fees_expense(),
        this->
            ticket_sale->
            event->
            revenue_account );

/* Output */
void ticket_refund_update(
        education_net_refund_amount(),
        ticket_refund_transaction()->
            transaction_date_time,
        event_name,
        event_date,
        event_time,
        payor_full_name,
        payor_street_address,
        refund_date_time );
```

TRANSACTION

Education Data Structure

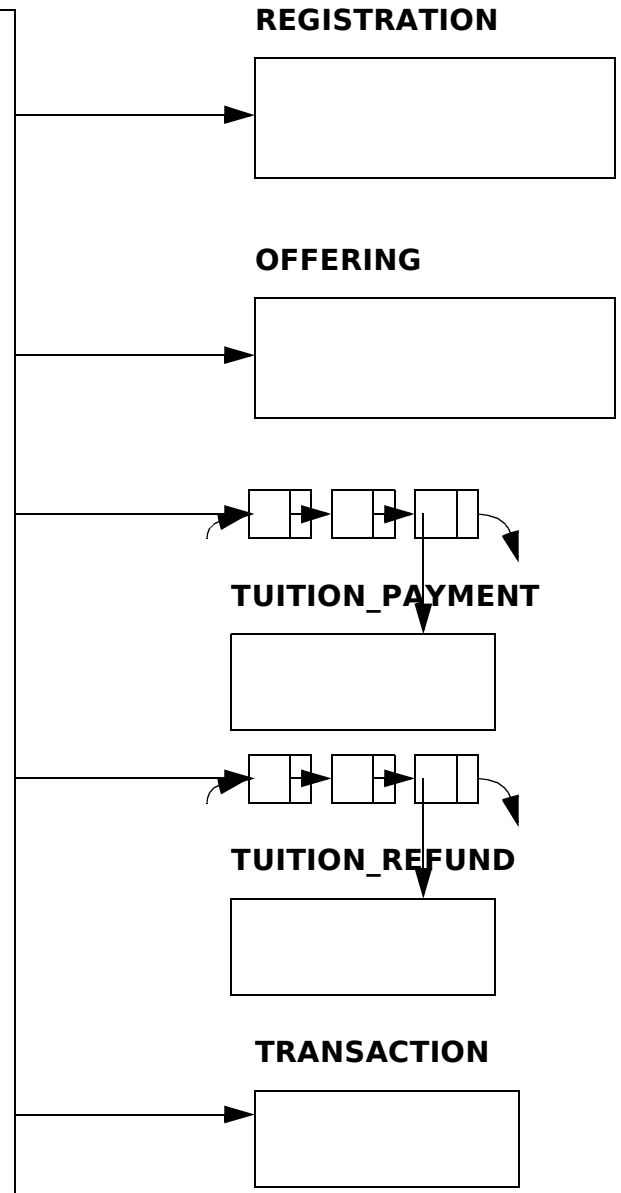
REGISTRATION

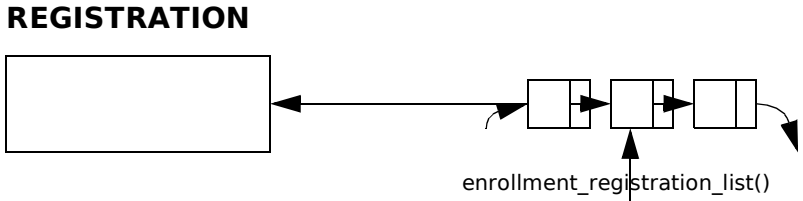
```
/* Usage */
REGISTRATION *registration_fetch(
                student_full_name,
                student_street_address,
                season_name,
                year,
                fetch_enrollment_list );

/* Input */
char *student_full_name;
char *street_address;
char *season_name;
int year;

/* Process */
LIST *registration_enrollment_list(
          student_full_name,
          street_address,
          season_name,
          year );

double registration_tuition(
          registration_enrollment_list(),
          semester_offering_list() );

LIST *registration_tuition_payment_list(
          registration_enrollment_list() );

double registration_tuition_payment_total(
          registration_tuition_payment_list )
{
          return sum(A->
                    tuition_payment->
                    payment_amount );
}

double registration_invoice_amount_due(
          registration_tuition(),
          registration_tuition_payment_total(),
          registration_tuition_refund_total() );

LIST *registration_tuition_refund_list(
          registration_enrollment_list() );

double registration_tuition_refund_total(
          registration_tuition_refund_list() );

void registration_update(
          registration_tuition(),
          registration_tuition_payment_total(),
          registration_invoice_amount_due(),
          student_full_name,
          student_street_address,
          season_name,
          year );
```



ENROLLMENT

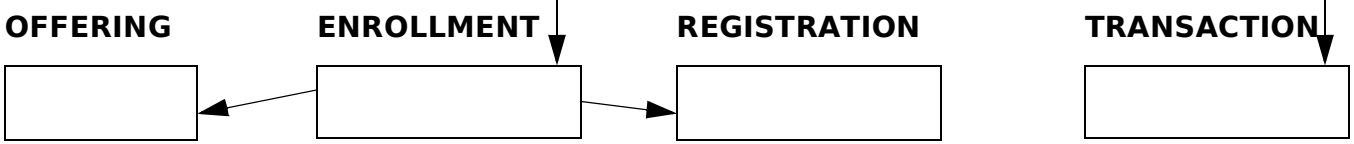
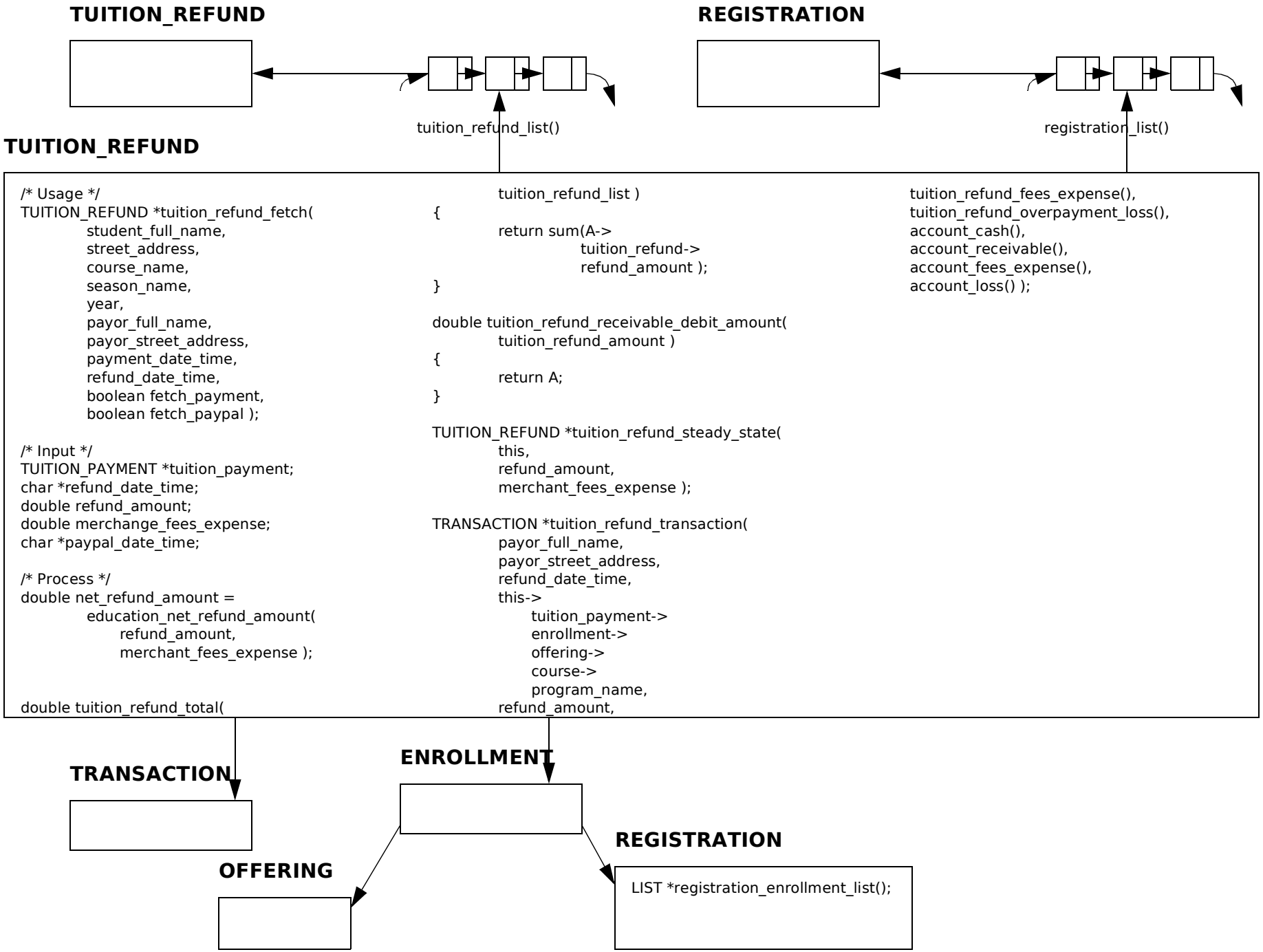
```
/* Usage */
ENROLLMENT *enrollment_fetch(
        student_full_name,
        street_address,
        course_name,
        season_name,
        year,
        fetch_tuition_payment_list,
        fetch_offering,
        fetch_registration,
        fetch tuition_refund_list );

/* Input */
LIST *enrollment_tuition_payment_list(
        student_full_name,
        street_address,
        course_name,
        season_name,
        year );

LIST *enrollment_tuition_refund_list(
        student_full_name,
        street_address,
        course_name,
        season_name,
        year );

OFFERING *offering =
        offering_new(
            course_name,
            season_name,
            year );

REGISTRATION *registration =
        registration_fetch(
            student_full_name,
            street_address,
            season_name,
            year,
            0 /* not fetch_enrollment_list */ );

/* Process */
double enrollment_tuition_payment_total(
        enrollment_tuition_payment_list() );

double enrollment_tuition_refund_total(
        enrollment_tuition_refund_list() );

char *enrollment_primary_where(
        student_full_name,
        street_address,
        course_name,
        season_name,
        year );

TRANSACTION *enrollment_transaction(
        student_full_name,
        street_address,
        this->registration->registration_date_time,
        this->offering->course->program_name,
        this->offering->course->course_price,
        account_receivable(),
        offering_revenue_account() );

void enrollment_cancelled_date_update(
        update_pipe,
        student_full_name,
        street_address,
        course_name,
        season_name,
        year,
        cancelled_date );
```





TUITION_PAYMENT

```
/* Usage */
TUITION_PAYMENT *tuition_payment_fetch(
        student_full_name,
        street_address,
        course_name,
        season_name,
        year,
        payor_full_name,
        payor_street_address,
        payment_date_time,
        boolean fetch_enrollment,
        boolean fetch_paypal );


/* Input */
ENROLLMENT *enrollment;
LIST *enrollment_registration_list;
double payment_amount;
double merchant_fees_expense;
char *paypal_date_time;

/* Process */
double net_payment_amount =
        education_net_payment_amount(
            payment_amount,
            merchant_fees_expense );

LIST *tuition_payment_list(
        this->
            enrollment->
                registration );

double tuition_payment_total(
        tuition_payment_list() )
{
        return sum(A->
                tuition_payment->
                payment_amount );
}

double tuition_payment_overpayment_donation(
        payment_amount,
        enrollment_registration_list() );

double tuition_payment_receivable_credit_amount(
        payment_amount )
{
        return A;
}

double tuition_payment_cash_debit_amount();

TUITION_PAYMENT *tuition_payment_steady_state(
        this,
        payment_amount,
        merchant_fees_expense );

TRANSACTION *tuition_payment_transaction(
        *transaction_seconds_to_add,
        payor_full_name,
        payor_street_address,
        payment_date_time,
        this->
            enrollment->
            offering->
            course->
            program->
            program_name,
        payment_amount,
        merchant_fees_expense,
        education_net_payment_amount(),
        tuition_payment_overpayment_donation(),
        account_cash(),
        account_receivable(),
        account_fees_expense(),
        account_overpayment_donation() );

void tuition_payment_update(
        education_net_payment_amount(),
        tuition_payment_overpayment_donation(),
        tuition_payment_transaction()->
            transaction_date_time,
        student_full_name,
        street_address,
        course_name,
        season_name,
        year,
        payor_full_name,
        payor_street_address,
        payment_date_time );
```



Education Data Structure